

AdvR 06 – Environments

Environment Basics

Four Key Differences from Lists

An environment is like a named list, with four exceptions:

- ① Every name must be **unique**
- ② Names are **not ordered**
- ③ Every environment has a **parent**
- ④ Environments have **reference semantics** (modify in place, not copy-on-modify)

Environments can even **contain themselves**:

```
e1 <- env(a = 1, b = 2)
e1$self <- e1
env_names(e1)
```

```
#> [1] "a"      "b"      "self"
```

Creating and Inspecting Environments

Create with `rlang::env()` (like `list()`):

```
e1 <- env(a = FALSE, b = "a", c = 2.3, d = 1:3)
e1    # just prints memory address
```

```
#> <environment: 0x5ebc0752bf10>
```

Use `env_print()` for useful info:

```
env_print(e1)
```

```
#> <environment: 0x5ebc0752bf10>
#> Parent: <environment: global>
#> Bindings:
#> * a: <lgl>
#> * b: <chr>
#> ... [output truncated]
```

```
env_names(e1)
```

```
#> [1] "a" "b" "c" "d"
```

Important Environments

- `current_env()`: where code is currently executing
- `global_env()`: the interactive workspace

Compare with `identical()`, `not ==`:

```
identical(global_env(), current_env())
```

```
#> [1] TRUE
```

```
global_env() == current_env()
```

```
#> Error in global_env() == current_env(): comparison (==) is possible only for atomic and list  
Base equivalents: globalenv(), environment().
```

Parents

Every environment has a **parent** (used for lexical scoping):

```
e2a <- env(d = 4, e = 5)
e2b <- env(e2a, a = 1, b = 2, c = 3)
env_parent(e2b)
```

```
#> <environment: 0x5ebc0a64a3f0>
```

The only environment without a parent is the **empty environment**:

```
env_parents(e2b)
```

```
#> [[1]]    <env: 0x5ebc0a64a3f0>
```

```
#> [[2]] $ <env: global>
```

```
env_parents(e2b, last = empty_env())
```

```
#> [[1]]    <env: 0x5ebc0a64a3f0>
```

```
#> [[2]] $ <env: global>
```

```
#> [[3]] $ <env: package:lobstr>
```

```
#> ... [output truncated]
```

Base equivalent: `parent.env()`.

<- creates a variable in the **current** environment. <<- modifies an existing variable in a **parent** environment:

```
x <- 0
f <- function() { x <<- 1 }
f()
x
```

```
#> [1] 1
```

If <<- doesn't find the variable, it creates one in the **global** environment — usually undesirable. Best used with function factories (closures).

Getting and Setting

Use `$` and `[[` like lists, but **no** numeric indexing or `:`:

```
e3 <- env(x = 1, y = 2)
```

```
e3$x
```

```
#> [1] 1
```

```
e3[["y"]]
```

```
#> [1] 2
```

```
e3[[1]]
```

```
#> Error in e3[[1]]: wrong arguments for subsetting an environment
```

```
env_get() errors on missing names (safer than $):
```

```
env_get(e3, "xyz")
```

```
#> Error in `env_get()`:
```

```
#> ! Can't find `xyz` in environment.
```


`env_bind()` sets multiple bindings at once:

```
env_bind(e3, a = 10, b = 20)  
env_names(e3)
```

```
#> [1] "x" "y" "a" "b"
```

Removing Bindings

Setting to NULL does **not** remove — use `env_unbind()`:

```
e3$a <- NULL  
env_has(e3, "a")  # still there!
```

```
#>      a  
#> TRUE
```

```
env_unbind(e3, "a")  
env_has(e3, "a")  # now gone
```

```
#>      a  
#> FALSE
```

Delayed bindings — evaluated only on first access (like promises):

```
env_bind_lazy(current_env(), b = {Sys.sleep(1); 1})  
system.time(print(b)) # slow (evaluates)  
system.time(print(b)) # fast (cached)
```

Used by `autoload()` to defer loading package datasets.

Active bindings — re-computed on every access:

```
env_bind_active(current_env(), z1 = function(val) runif(1))  
c(z1, z1, z1) # different each time
```

```
#> [1] 0.18259971 0.01533119 0.58618790
```

Used by R6 active fields.

Exercises (Basics)

- 1 List three ways an environment differs from a list.
- 2 Explain why `e[[1]]` and `e[c("a", "b")]` don't make sense for environments.
- 3 What does this function do? How does it differ from `<<-`?

```
rebind <- function(name, value, env = caller_env()) {  
  if (identical(env, empty_env())) {  
    stop("Can't find `", name, "`", call. = FALSE)  
  } else if (env_has(env, name)) {  
    env_poke(env, name, value)  
  } else {  
    rebind(name, value, env_parent(env))  
  }  
}
```

- ❶ Key differences:
 - **Reference semantics** (no copy-on-modify)
 - Every environment has a **parent**
 - All names must be **unique**
 - Names are **not ordered**
 - Can contain themselves
- ❷ `e[[1]]` — elements are not ordered, so numeric indexing is meaningless. `e[c("a", "b")]` — `[` is not defined for environments; subsetting would need to return something, but environments are not subsettable structures.

- ③ `rebind()` walks up parent environments looking for an existing binding to reassign. Unlike `<-`, it **never creates** a new variable — it throws an error if the name doesn't exist anywhere. This is safer because `<-` silently creates global variables when no binding is found.

```
a <- 5
rebind("a", 10)
a
```

```
#> [1] 10
```

```
rebind("zzz", 10) # error: doesn't exist
```

```
#> Error: Can't find `zzz`
```

Recursing over Environments

The where() Function

A template for **recursive environment traversal**:

```
where <- function(name, env = caller_env()) {  
  if (identical(env, empty_env())) {  
    stop("Can't find ", name, call. = FALSE)  
  } else if (env_has(env, name)) {  
    env  
  } else {  
    where(name, env_parent(env))  
  }  
}  
  
x <- 5  
where("x")
```

```
#> <environment: R_GlobalEnv>
```

```
where("mean")
```

```
#> <environment: base>
```


Recursive Template

The general pattern for computing over environments:

```
f <- function(..., env = caller_env()) {  
  if (identical(env, empty_env())) {  
    # base case  
  } else if (success) {  
    # success case  
  } else {  
    # recursive case  
    f(..., env = env_parent(env))  
  }  
}
```

Can also use iteration:

```
f <- function(..., env = caller_env()) {  
  while (!identical(env, empty_env())) {  
    if (success) return()  
    env <- env_parent(env)  
  }  
  # base case  
}
```

Exercises (Recursion)

- 1 Modify `where()` to return **all** environments that contain a binding for `name`.
- 2 Write `fget()` that finds only **function** objects, skipping non-function bindings with the same name.

- 1 Recurse to the empty environment regardless of matches; collect all hits:

```
where2 <- function(name, env = caller_env(), results = list()) {  
  if (identical(env, empty_env())) {  
    results  
  } else {  
    if (env_has(env, name)) results <- c(results, env)  
    where2(name, env_parent(env), results)  
  }  
}
```

- ② Check if the found object is a function; if not, recurse:

```
fget <- function(name, env = caller_env()) {  
  if (env_has(env, name)) {  
    obj <- env_get(env, name)  
    if (is.function(obj)) return(obj)  
  }  
  if (identical(env, empty_env()))  
    stop("No function called \"", name, "\", call. = FALSE)  
  fget(name, env_parent(env))  
}  
  
mean <- 10  
fget("mean") # finds base::mean, not the value 10
```

```
#> function (x, ...)  
#> UseMethod("mean")  
#> <bytecode: 0x5ebc08159590>  
#> ... [output truncated]
```

Special Environments

Package Environments and Search Path

Each `library()` call adds a package to the **search path** — a chain of parent environments above global:

```
search()
```

```
#> [1] ".GlobalEnv"      "package:lobstr"  
#> [3] "package:rlang"    "package:stats"  
#> [5] "package:graphics" "package:grDevices"  
#> [7] "package:utils"    "package:datasets"  
#> [9] "package:methods"  "Autoloads"  
#> ... [output truncated]
```

- **Last attached** = immediate parent of global
- Bottom of chain: `Autoloads` → `package:base`

The search path is why you can call `mean()` without `base::mean()`.

A function captures the environment where it was **defined** — its **function environment** (= enclosing environment):

```
y <- 1
f <- function(x) x + y
fn_env(f)
```

```
#> <environment: R_GlobalEnv>
```

This is why R uses **lexical scoping**: the function looks up `y` where it was defined, not where it was called.

Functions that capture their environment are called **closures**.

Base equivalent: `environment(f)`.

Problem: If the search path determines lookups, wouldn't different `library()` orders break packages?

Solution: namespaces. Every package has two environments:

- **Package environment** — external interface (what users see via `::`)
- **Namespace environment** — internal interface (how functions find variables)

The namespace chain: `imports` → base namespace → global env.

This ensures `sd()` always finds `var()` from `stats`, regardless of what's on the search path.

Each function call creates a **fresh execution environment**:

```
g <- function(x) {  
  if (!env_has(current_env(), "a")) {  
    message("Defining a"); a <- 1  
  } else a <- a + 1  
  a  
}  
c(g(10), g(10))  # both 1 (fresh start)
```

```
#> [1] 1 1
```

- Parent of execution env = **function environment**
- Usually **ephemeral** (garbage collected after return)
- Persists if you return it or return a closure that captures it

Returning a function captures the execution environment:

```
plus <- function(x) {  
  function(y) x + y  
}  
plus_one <- plus(1)  
plus_one(2)
```

```
#> [1] 3
```

The enclosing environment of `plus_one` is the (now-persistent) **execution environment** of `plus(1)`, which binds `x = 1`.

- 1 How is `search_envs()` different from `env_parents(global_env())`?
- 2 What is the enclosing environment of each nested function?

```
f1 <- function(x1) {  
  f2 <- function(x2) {  
    f3 <- function(x3) { x1 + x2 + x3 }  
    f3(3)  
  }  
  f2(2)  
}
```

`f1(1)`

- ① `search_envs()` includes the global env and all attached packages (ends at base).
`env_parents(global_env())` excludes the global env itself and includes the empty env at the end.
- ② `f1` is defined in the global env → encloses global. `f2` is defined inside `f1`'s execution → encloses that execution env. `f3` is defined inside `f2`'s execution → encloses that execution env. Each nested function captures variables from all enclosing scopes (`x1`, `x2`, `x3`).

Call Stacks

Call Stacks and Frames

The **call stack** tracks the sequence of function calls:

```
f <- function(x) g(x = 2)
g <- function(x) h(x = 3)
h <- function(x) lobster::cst()
```

```
f(x = 1)
#> x
#> +-f(x = 1)
#> | \-g(x = 2)
#> |   \-h(x = 3)
#> |       \-lobster::cst()
```

Each element is a **frame**: an expression, an environment, and a parent frame.

The **caller environment** (`caller_env()` or base `parent.frame()`) is the environment from which the function was **called**.

Lexical scoping: look up values where the function was **defined**.

Dynamic scoping: look up values where the function was **called**.

R uses lexical scoping by default. Dynamic scoping is used for:

- Interactive data analysis tools (tidy evaluation)
- `parent.frame()` / `caller_env()` patterns

Few languages use dynamic scoping (Emacs Lisp is a notable exception).

- 1 Write a function that lists all variables defined in the environment in which it was **called** (same result as `ls()`).

Solutions (Call Stacks)

- 1 Use `caller_env()` to access the calling environment:

```
ls2 <- function(env = caller_env()) {  
  sort(env_names(env))  
}  
e1 <- env(a = 1, b = 2)  
ls2(e1)
```

```
#> [1] "a" "b"
```

Environments as Data Structures

Three Use Cases

Environments have **reference semantics** — useful beyond scoping:

1. Avoiding copies of large data — no accidental copy-on-modify. Better yet, use R6 objects (built on environments).

2. Managing state within a package:

```
my_env <- new.env(parent = emptyenv())
my_env$a <- 1
get_a <- function() my_env$a
set_a <- function(value) {
  old <- my_env$a; my_env$a <- value
  invisible(old)
}
```

3. As a hashmap — $O(1)$ lookup by name (constant time regardless of size).